

Memoria técnica del proyecto mgcss-track-grupo8

Entrega final · Grupo 8

Realizado por:

- Jose Manuel Roldán Ruiz
- Jose Delgado García
- Antonio Pablo García Sanabria

1. Introducción

Este documento recoge la memoria técnica del proyecto **mgcss-track-grupo8**. El objetivo es describir de forma estructurada la solución implementada, las decisiones técnicas tomadas y el proceso seguido hasta completar los requisitos obligatorios de la entrega.

2. Contexto y objetivos

El proyecto se ha desarrollado sobre una aplicación basada en Spring Boot, orientada a la gestión de incidencias y a la organización de su tratamiento dentro del dominio funcional del sistema.

Los objetivos principales han sido consolidar una versión estable mediante etiquetado semántico, automatizar la generación de releases a partir de tags del repositorio, garantizar la trazabilidad entre commit, tag, workflow y artefacto final, y documentar de forma clara el proceso seguido durante la práctica.

3. Herramientas utilizadas y papel en la arquitectura

Spring Tools Suite sobre Eclipse ha sido nuestro entorno principal de desarrollo. Lo hemos usado para escribir el código, organizar los paquetes, ejecutar la aplicación y depurar errores. Esta herramienta afecta sobre todo a la parte de aplicación y lógica de negocio, porque es donde hemos trabajado con controladores, servicios, repositorios, DTOs y configuración general del backend.

Maven ha sido la herramienta encargada de la construcción del proyecto y de la gestión de dependencias. Gracias al pom.xml hemos centralizado la configuración de librerías, la compilación, la ejecución de tests y la generación del archivo .jar. En la práctica, Maven toca la parte de construcción y empaquetado del proyecto.

GitHub ha sido el repositorio central del proyecto y el lugar donde hemos controlado versiones, ramas, tags y releases. Esta parte se relaciona con la gestión del código fuente y con la trazabilidad de la evolución del proyecto.

GitHub Actions ha cubierto la parte de automatización del ciclo de vida. En nuestro caso, hemos definido un workflow de release que se lanza al hacer push de un tag y que compila el proyecto, genera el artefacto, construye la imagen Docker y publica la release.

Docker ha sido la herramienta utilizada para generar una imagen de la aplicación. Esto nos ha permitido preparar una capa de despliegue más cercana a un entorno real y separar la ejecución de la aplicación del entorno local de desarrollo.

Por último, H2 ha sido la base de datos ligera utilizada durante el desarrollo y las pruebas. Nos ha permitido trabajar con persistencia sin depender de una base de datos externa, lo que ha

simplificado bastante la fase inicial del proyecto.

4. Organización del trabajo

El trabajo se ha abordado como un proyecto de grupo, distribuyendo las tareas de manera que se pudiera avanzar tanto en la parte funcional de la aplicación como en la automatización de la entrega final.

Durante el desarrollo se han combinado tareas de programación, revisión del repositorio, configuración de dependencias, depuración del workflow y validación final del proceso de release. La coordinación del grupo ha sido importante especialmente en la fase final, ya que varios problemas detectados no estaban en la lógica de negocio, sino en la configuración de Maven, las dependencias y la automatización en GitHub.

5. Automatización del ciclo de release

Uno de los puntos más importantes de la entrega final ha sido la definición de un workflow específico de release dentro de `.github/workflows/release.yml`. GitHub establece que un workflow es un proceso automatizado configurable y que debe definirse mediante un archivo YAML en el repositorio.

En nuestro caso, el workflow se ha configurado para activarse al hacer push de tags. A partir de ese evento, el pipeline ejecuta varias etapas consecutivas: descarga del código, configuración de Java, compilación del proyecto con Maven, copia y renombrado del artefacto `.jar`, construcción de la imagen Docker y creación automática de la release en GitHub adjuntando el artefacto generado.

Esta automatización reduce tareas manuales repetitivas, mejora la trazabilidad del proceso y deja el repositorio preparado para futuras iteraciones del proyecto con una mecánica de publicación más robusta.

6. Verificación y trazabilidad

Tras definir el workflow, se realizó una verificación práctica creando nuevos tags sobre commits recientes para comprobar el comportamiento del pipeline. Esta comprobación era necesaria porque el tag inicial `v1.0.0` se había creado antes de incorporar el workflow, por lo que no servía para validar retroactivamente la automatización nueva.

Durante las pruebas se detectó que el problema principal no estaba en el contenido del workflow, sino en la manera en que se estaban generando y haciendo el push de los tags desde el entorno de desarrollo. Una vez corregido este punto, el workflow se ejecutó correctamente y la ejecución apareció en la pestaña de Actions del repositorio, terminando en verde.

Con ello se pudo verificar la trazabilidad completa entre los distintos elementos del proceso: el commit contiene el cambio validado, el tag identifica una versión concreta del proyecto, el push del tag activa el workflow de release, el workflow compila y genera el artefacto, y la release queda publicada en GitHub con el `.jar` adjunto.

7. Incidencias técnicas encontradas

A lo largo del trabajo aparecieron varias incidencias técnicas. Una de ellas estuvo relacionada con la resolución de dependencias Maven y el uso de `jakarta.validation`, ya que en el entorno de desarrollo aparecían errores de importación en clases DTO aun existiendo la dependencia de validación en el `pom.xml`.

También tuvimos incidencias con el pom.xml, porque al revisar las dependencias vimos que algunas estaban poco alineadas con una configuración estándar de Spring Boot. Eso nos llevó a revisar mejor qué dependencias eran realmente necesarias y cuáles podían estar generando incompatibilidades o comportamientos extraños.

Otro punto que nos dio trabajo fue la validación del DTO con Jakarta Validation, porque aunque el código era correcto, el IDE no resolvía bien el import al principio. Esto nos hizo ver que en un proyecto real no basta con escribir bien la clase; también hay que asegurarse de que toda la cadena de herramientas está bien configurada.

8. Instrucciones de instalación

Antes de empezar, es necesario tener instalado Java 17 o superior y contar con una IDE con soporte para Spring, como Spring Tools Suite o Eclipse.

1. Clonar el repositorio. Descargamos la práctica desde GitHub con ``git clone`` y entramos en la carpeta del proyecto.
2. Importar el proyecto en la IDE. Lo abrimos como un proyecto Maven existente para que se descarguen automáticamente las dependencias definidas en el ``pom.xml``.
3. Revisar la configuración de Java. Comprobamos que el proyecto está usando Java 17 tanto en el compilador como en el build path.
4. Ejecutar la aplicación. Desde la IDE se puede lanzar con ``Run As > Spring Boot App``. También se puede ejecutar con ``mvn spring-boot:run``.
5. Verificar que funciona correctamente. Si la aplicación expone consola H2 o una interfaz web, accedemos a ``localhost`` para comprobar que arranca sin errores y que la persistencia está activa.
6. Generar el artefacto. Si se quiere comprobar el empaquetado, ejecutamos ``mvn clean package`` para crear el ``jar`` en la carpeta ``target/``.
7. Probar Docker. Si se quiere comprobar la versión contenedorizada, se construye la imagen Docker y se verifica que la aplicación arranca correctamente dentro del contenedor.

9. Conclusiones

En nuestra opinión, una parte de las prácticas ha estado bien porque hemos tocado varias técnicas que nos vendrán bien en un futuro porque son técnicas que se utilizan hoy en día. También está el tema del trabajo en equipo con Github de por medio, es algo que creemos que nos ha venido bien por la organización que hemos tenido que llevar a cabo entre nosotros.

Por otro lado, creemos que se ha notado un poco de descontrol en las prácticas ya que parecía que se iban reestructurando a medida que pasaba el tiempo, es decir, no veíamos una guía fija. Las prácticas en general han estado muy bien pero hay matices que se deberían cambiar.